Pseudorandom Generators & Pseudorandom Functions

ACM Summer School on Symmetric Key Cryptography, Day 2

June 9, 2026

Limitations of Information-Theoretic Security

- **One-Time Pad (OTP) :** Ideal scenario since it ensures perfect secrecy i.e., $P[M = m \mid C = c] = P[M = m]$.
- **Shannon's Theorem:** Perfect secrecy strictly requires that the key space size must be greater than or equal to the message space size ($|\mathcal{K}| \geq |\mathcal{M}|$).
- **Not practical!** In real world applications, securely exchanging and storing truly random keys of length equal to total transmitted data is completely unfeasible.
- **Solution:** Relaxing security from *information-theoretic* (attacker with infinite resources) to *computational security* (bounded polynomial-time attacker).

We call this bounded polynomial time attacker as a *Probabilistic Polynomial Time/ PPT adversary*. To be fair, we restrict Alice and Bob to be PPT algorithms too.

What is Semantic/Computational Security?

An encryption scheme is **semantically secure** if the ciphertext does not leak *any* structural or textual information about the underlying plaintext to a computationally bounded adversary.

- Trying to prove that an attacker "cannot recover the entire key" is too weak.
- Attacking a system might just mean figuring out a single bit (e.g., if a transaction is a "Buy" or "Sell" order).
- Regardless of the attacker's prior knowledge, viewing a ciphertext should provide **zero additional advantage**.

Indistinguishability Approach

Instead of guessing data, we challenge the adversary to distinguish between the encryptions of two distinct, user-chosen messages.

Stream Ciphers

The Key Idea

Replace the infinitely long, truly random key of the One-Time Pad with a long, **pseudorandom** string generated deterministically from a short, truly random seed.

A stream cipher consists of an encryption algorithm E and a decryption algorithm D mapping over spaces $(\mathcal{K}, \mathcal{M}, \mathcal{C})$:

- **Seed/Key Space:** $\mathcal{K} = \{0,1\}^s$ (where s is computationally secure, e.g., 128 or 256 bits).
- **Keystream Expansion Generator (G):** $G : \{0,1\}^s \to \{0,1\}^n$, where $n \gg s$.
- **Encryption:**

$$E(k, m) := m \oplus G(k) = c$$

- **Decryption:**

$$D(k, c) := c \oplus G(k) = m$$

Formalizing Pseudorandom Generators (PRGs)

Let $G : \{0,1\}^s \to \{0,1\}^n$ be a deterministic, polynomial-time computable function where $n > s$.

- The input space is sampled uniformly at random: $k \leftarrow \{0,1\}^s$.
- The output space is a tiny subset of the absolute space: Co-domain$(G) \subsetneq \{0,1\}^n$. Specifically, $|\text{Codomain}(G)| = 2^s$, whereas $|\{0,1\}^n| = 2^n$.
- **Core Question:** Can a probabilistic polynomial-time (PPT) adversary distinguish a string sampled uniformly from Codomain(G) from a string sampled uniformly from $\{0,1\}^n$?

Attack Game (PRG)

- How do we mathematically state that a deterministic string looks *random* to a computer?
- We use a Challenger-Adversary Game. Instead of trying to prove a string has zero patterns, we prove that no realistic algorithm can *find* a pattern.
- To formalize security, we introduce a PRG adversary $\mathcal{A}$ modeled as a PPT algorithm working in a controlled simulation managed by a *Challenger.*

PRG Construction

Let $G : \mathcal{K} \to \{0,1\}^n$ be a deterministic, polynomial-time algorithm where:

- $\mathcal{K} = \{0,1\}^s$ is the seed space (seed of length s).
- $\{0,1\}^n$ is the output space (of length n).
- The expansion factor strictly requires $n > s$ (often $n \gg s$).

PRG Attack Game

Let $G : \{0,1\}^s \to \{0,1\}^n$ be a PRG, and let $\mathcal{A}$ be a PPT adversary.

1. The Challenger $\mathcal{C}$ chooses a hidden challenge bit uniformly at random: $b \leftarrow \{0,1\}$.
2. $\mathcal{C}$ prepares a challenge string $w \in \{0,1\}^n$ based on b:
   - **If** $b = 1$**:** Choose a random seed $k \leftarrow \{0,1\}^s$ and compute $w \leftarrow G(k)$.
   - **If** $b = 0$**:** Choose a truly random string $w \leftarrow \{0,1\}^n$.
3. $\mathcal{C}$ sends the string w to the Adversary $\mathcal{A}$.
4. $\mathcal{A}$ processes w and outputs a guess bit $\hat{b} \in \{0,1\}$.

The adversary wins PRG Attack Game if its final guess matches the challenger's hidden choice: $\hat{b} = b$.

PRG Attack Game – Explanation

We can split the attack game into two depending on the value of b:

Experiment $(b = 1)$ - PRG Choice

- Seed sampled: $k \leftarrow \{0,1\}^s$
- Vector derived: $w = G(k)$
- $\mathcal{A}$ is given a string calculated deterministically from a short random seed.

$$\Pr[\mathcal{A}(G(k)) = 1]$$

Experiment $(b = 0)$ - Random Choice

- Vector sampled: $r \leftarrow \{0,1\}^n$
- Vector derived: $w = r$
- $\mathcal{A}$ is given a string generated by a perfect physical source of uniform chaos.

$$\Pr[\mathcal{A}(r) = 1]$$

Defining $\mathbf{Adv}_{\mathsf{PRG}}[G, \mathcal{A}]$

Clearly any adversary can win 50% of the time just by guessing randomly ($\hat{b} \leftarrow \{0,1\}$), we need to know what is advantage better than $\frac{1}{2}$.

Definition (PRG Advantage)

The formal advantage of an adversary $\mathcal{A}$ against a PRG G in PRG Attack Game is defined as:

$$\mathbf{Adv}_{\mathsf{PRG}}[G, \mathcal{A}] := \left| \Pr[\hat{b} = 1 \mid b = 1] - \Pr[\hat{b} = 1 \mid b = 0] \right|$$

Expanding this definition using the above two experiments yields:

$$\mathbf{Adv}_{\mathsf{PRG}}[G, \mathcal{A}] = \left| \Pr_{k \leftarrow \{0,1\}^s}[\mathcal{A}(G(k)) = 1] - \Pr_{r \leftarrow \{0,1\}^n}[\mathcal{A}(r) = 1] \right|$$

Super-Polynomial Growth and Reciprocals

Defining Super-Polynomial Functions

A function $f(\lambda)$ is called **super-polynomial** if it grows asymptotically faster than *any* polynomial. Formally, for every positive polynomial $p(\lambda)$, there exists λ_0 such that for all $\lambda > \lambda_0$:

$$|f(\lambda)| > p(\lambda) \quad \iff \quad \lim_{\lambda \to \infty} \frac{p(\lambda)}{f(\lambda)} = 0$$

Dual Reciprocality

Note that a function $f(\lambda)$ is super-polynomial **if and only if its reciprocal** $1/f(\lambda)$ **is negligible**.

Why This Matters for Adversarial Success Bounds

Suppose an adversary repeats a basic cracking strategy $p(\lambda)$ times (where p is any polynomial bound on their running time). If the single-try success chance is bounded by a negligible function $1/f(\lambda)$, their net success chance after a lifetime of efforts is bounded by $\frac{p(\lambda)}{f(\lambda)}$. Because $f(\lambda)$ is super-polynomial, $\lim_{\lambda \to \infty} \frac{p(\lambda)}{f(\lambda)} = 0$. The attacker cannot win, even with full polynomial-time resource amplification.

Security of a PRG

How small must the advantage be for us to consider a PRG secure?

Asymptotic Security

Let λ be the security parameter (the seed length). A generator G is a secure PRG if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $negl(\lambda)$ such that:

$$\mathbf{Adv}_{\mathrm{PRG}}[G, \mathcal{A}] \leq negl(\lambda)$$

Recall that $negl(\lambda)$ drops faster than the reciprocal of any polynomial $(1/\lambda^c)$, where λ is the security parameter.

Connecting the Attack Game to the Security of Stream Ciphers

We want to prove that if a PRG passes the above game, the stream cipher built from it ($c = m \oplus G(k)$) is safe from passive eavesdroppers.

The Semantic Security Game

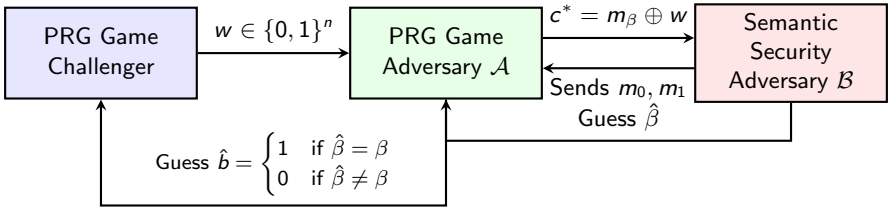
An adversary chooses two equal-length messages, m_0 and m_1. The challenger encrypts one ($c^* = m_\beta \oplus G(k)$). The adversary must guess which message was encrypted ($\hat{\beta}$).

Theorem (Foundational Security Reduction)

If a PRG G is secure as per the PRG game, then the corresponding stream cipher provides semantic security against passive eavesdroppers.

The Security Proof Reduction

We use a proof by contradiction. Suppose an attacker $\mathcal{B}$ can break the semantic security of the cipher. We will build an adversary $\mathcal{A}$ that uses $\mathcal{B}$ as a tool to win the PRG Game.



If $\mathcal{B}$ can spot patterns in the cipher, $\mathcal{A}$ will leverage that to spot patterns in the PRG string w.

The Reduction Algorithm

Here is exactly how our constructed adversary $\mathcal{A}$ plays the PRG Game:

Receive challenge string $w \in \{0,1\}^n$ from the PRG Game Challenger;
Initialize the semantic security attacker $\mathcal{B}$;
Receive two target plaintexts $m_0, m_1 \in \{0,1\}^n$ from $\mathcal{B}$;
Sample a random bit internally: $\beta \leftarrow \{0,1\}$;
Construct a simulated ciphertext: $c^* \leftarrow m_\beta \oplus w$;
Send c^* to $\mathcal{B}$ and collect its guess $\hat{\beta}$;

if $\hat{\beta} == \beta$ **then**
  return 1 ; // Guessing w is a real PRG output ($b = 1$)
end
else
  return 0 ; // Guessing w is truly random ($b = 0$)
end

Algorithm 1: PRG Game Adversary $\mathcal{A}(w)$

Analysis (The Random Case)

Case 0: The challenger picked $b = 0$ (w is a truly random string r)

- The simulated ciphertext becomes: $c^* = m_\beta \oplus r$.
- Because r is a truly uniform random string, XORing it with m_β acts as a One-Time Pad.
- This means c^* is completely independent of the message index β. It leaks zero information, even to an computationally unbounded attacker.
- Therefore, $\mathcal{B}$ has absolutely no edge and must guess blindly:

$$\Pr[\hat{\beta} = \beta \mid b = 0] = \frac{1}{2}$$

- Looking at our algorithm, $\mathcal{A}$ outputs 1 only when $\hat{\beta} = \beta$. Thus:

$$\Pr[\mathcal{A}(r) = 1] = \frac{1}{2}$$

Analysis (The PRG Case)

Case 1: The challenger picked $b = 1$ (w is a pseudorandom string $G(k)$)

- The simulated ciphertext becomes: $c^* = m_\beta \oplus G(k)$.
- This matches the actual stream cipher encryption scheme.
- Because this simulation is perfect, $\mathcal{B}$'s probability of winning matches its real-world semantic security success rate: $\Pr[\hat{\beta} = \beta \mid b = 1] = \mathsf{PrivK}^{eav}_{\mathcal{A},E}(1^n) + \frac{1}{2}$.
- Since $\mathcal{A}$ outputs 1 whenever $\mathcal{B}$ is correct, we have:

$$\Pr[\mathcal{A}(G(k)) = 1] = \mathsf{PrivK}^{eav}_{\mathcal{A},E}(1^n) + \frac{1}{2}$$

Combining the Results

$$\begin{aligned}\mathbf{Adv}_{\mathsf{PRG}}[G,\mathcal{A}] &= |\Pr[\mathcal{A}(G(k))=1] - \Pr[\mathcal{A}(r)=1]| \\ &= \left|\mathsf{PrivK}^{eav}_{\mathcal{A},E}(1^n) + \frac{1}{2} - \frac{1}{2}\right|\end{aligned}$$

Security Result

If the semantic security attacker $\mathcal{B}$ can manage a non-negligible advantage, our distinguisher $\mathcal{A}$ will instantly achieve a non-negligible advantage in PRG Game. Thus, if a PRG is secure the stream cipher is guaranteed to be semantically secure.

Hybrid Argument

- PRG Gameis often used to prove things about complex structures through a technique called the *Hybrid Argument*.
- Suppose we chain a PRG together to expand a seed by multiple bits step-by-step. How do we prove the final long string is secure?
- We define a series of intermediate games (*hybrids*) $H_0, H_1, \ldots, H_m$:
  - In H_0, the string is completely pseudorandom (PRG Game with $b = 1$).
  - In each successive hybrid H_i, we replace the next chunk of the string with truly random bits.
  - In the final hybrid H_m, the entire string is completely random (PRG Game with $b = 0$).
- If an attacker can tell the difference between the first and last game, it must be able to tell the difference between at least two adjacent intermediate games (H_i and H_{i+1}).
- This single step difference maps perfectly back to a single instance of the PRG Attack Game.

Hybrid arguments are an important technique in cryptography that can be used to scale seamlessly – allowing us to build complex, multi-layered cryptographic protocols from simple base components.

Introduction to Composition Theories

- Often, foundational primitives or historical assumptions yield base PRGs that only extend a seed by a tiny amount (e.g., $G : \{0,1\}^s \to \{0,1\}^{s+1}$ stretching by just 1 bit).
- Build rigorous, modular methods to chain these minor generators together to achieve an arbitrary polynomial length expansion ($n = \text{poly}(s)$).
- **Security Question:** Does chaining distinct pseudorandom steps still preserve computational indinstinguishability?

Parallel vs. Sequential Composition

Let $G_1, G_2 : \{0,1\}^s \to \{0,1\}^n$ be secure PRGs.

Parallel Composition (Independent Seeds)

Define $G_{\text{par}}(k_1, k_2) := G_1(k_1) \parallel G_2(k_2)$.

- Input seed size doubles ($2s$), output layout doubles ($2n$). Security bounds cleanly split via simple coordinate isolation.

Sequential Composition

What happens if we feed parts of a PRG output as the seed into another PRG?

$$G_{\text{seq}}(k) := G_2(\text{Truncate}(G_1(k)))$$

- Requires more detailed proofs.

The 1-Bit Stretch Amplification Theorem

Theorem

If there exists a PRG $G_0 : \{0,1\}^s \to \{0,1\}^{s+1}$ that stretches its input by a single bit, then there exists a PRG $G_{ext} : \{0,1\}^s \to \{0,1\}^{s+m}$ stretching by any arbitrary polynomial number of bits m.

- This implies that the transition from a minimal output advantage to an expansive keystream is theoretically continuous.
- This construction provides the foundation for sequential composition

Introduction to the Blum-Micali Method

Objective

Take a Pseudo-Random Generator (PRG) that stretches a seed just a **little** bit, and build a new PRG that stretches it by an **arbitrary** amount.

- **Base PRG (G):** Defined over the domain and co-domain $(S, R \times S)$, for some finite sets S (seed space) and R.
- **Extended PRG (G'):** For any poly-bounded value $n \geq 1$, we construct a new PRG defined over $(S, R^n \times S)$.
- **Nomenclature:** This approach is known as the *n-wise sequential composition* of G.

The Sequential Construction Algorithm

For an initial seed $s \in S$, the extended generator $G'(s)$ evaluates as follows:

Algorithm: $G'(s)$

$s_0 \leftarrow s$
for $i = 1$ **to** n **do:**
  $(r_i, s_i) \leftarrow G(s_{i-1})$
output $(r_1, \ldots, r_n, s_n)$

- Each iteration consumes the previous internal state s_{i-1}.
- It outputs a fresh random block r_i and updates the next internal state s_i.

Schematic Description for $n = 3$ (Figure 3.6)

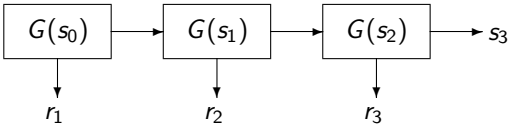


Figure: Visualizing the chain of iterations for $n = 3$ from Figure 3.6 of Boneh - Shoup textbook.

- Shows how the seed state snakes through sequentially.
- Outputs (r_1, r_2, r_3) are collected along the way alongside the final state s_3.

Security Theorem

Theorem (Theorem 3.3)

If G is a secure PRG, then the n-wise sequential composition G' of G is also a secure PRG.

Advantage Relation

For every PRG adversary A playing Attack Game 3.1 against G', there exists a PRG adversary B playing Attack Game 3.1 against G such that:

$$\mathrm{PRGadv}[A, G'] = n \cdot \mathrm{PRGadv}[B, G]$$

where B acts as an elementary wrapper around A.

Proof Intuition: Bridging the Gap

We want to show that an adversary A cannot distinguish real outputs from random strings using a **hybrid argument**:

- **Step 1:** In the real game (s_0 random), G's security means the first output pair $(r_1, s_1) \leftarrow G(s_0)$ can be replaced by a *completely random* element of $R \times S$. A's advantage changes negligibly.
- **Step 2:** Now that s_1 is truly random, the next pair $(r_2, s_2) \leftarrow G(s_1)$ can also be replaced by a *completely random* element.
- **Induction:** We incrementally replace (r_3, s_3) through (r_n, s_n) with random items.
- **Endpoint:** After n steps, all components are purely random (equivalent to Experiment 1). A's total output behavior remains negligibly close.

Schematic of Challengers for $n = 3$ (Figure 3.7)

Game	**How the string (r_1, r_2, r_3, s_3) is generated**
Hybrid_0	$(r_1, s_1) \leftarrow G(s_0),\ (r_2, s_2) \leftarrow G(s_1),\ (r_3, s_3) \leftarrow G(s_2)$
Hybrid_1	$r_1 \xleftarrow{R} R, s_1 \xleftarrow{R} S,\ (r_2, s_2) \leftarrow G(s_1),\ (r_3, s_3) \leftarrow G(s_2)$
Hybrid_2	$r_1, r_2 \xleftarrow{R} R, s_2 \xleftarrow{R} S,\ (r_3, s_3) \leftarrow G(s_2)$
Hybrid_3	$r_1, r_2, r_3 \xleftarrow{R} R, s_3 \xleftarrow{R} S$

Figure: Visualizing Figure 3.7 in Boneh-Shoup textbook

- Moving down the table, we replace one real application of G with a completely random output layer at each step.

Evaluation: Design Trade-offs

Expansion Rate Metric

If a PRG stretches an n-bit seed to an m-bit output, its expansion rate is m/n. More generally, for seed space S and output space R:

$$\text{Expansion Rate} = \frac{\log |R|}{\log |S|}$$

- **The Advantage:** The sequential composition method achieves a significantly higher and better expansion rate than parallel composition strategies.
- **The Core Drawback:** It cannot be parallelized. Every state update step i strictly requires the generation of s_{i-1} from the immediately preceding round.

The Two-Time Pad Catastrophe

The most severe real-world exploit vector against stream ciphers stems from operational key reuse.

- Suppose an operator encrypts two separate documents using the exact same key/seed state (k):

$$c_1 = m_1 \oplus G(k), \quad c_2 = m_2 \oplus G(k)$$

.

- An eavesdropper intercepting both ciphertexts can compute:

$$c_1 \oplus c_2 = (m_1 \oplus G(k)) \oplus (m_2 \oplus G(k)) = m_1 \oplus m_2$$

.

- The attacker is left with the direct XOR combination of both plaintexts.

Initialization Vectors (IVs)

To securely encrypt multiple messages under a single long-term secret key, the underlying PRG must accept two distinct inputs: a key and a unique *Initialization Vector (IV)* or *Nonce*.

$$G(k, IV) \rightarrow \{0,1\}^n$$

Security Commitments & Requirements

- The IV does **not** need to be secret. It is explicitly transmitted alongside the ciphertext.
- The pair (k, IV) must be unique across all messages. For a fixed key k, an IV must absolutely never be repeated.

Stateful vs. Randomized Nonces

Systems typically use one of two main strategies to ensure nonce uniqueness:

Stateful / Counter Approach

- Maintain an internal persistent counter.
- Increment the counter by 1 for each new message sent.
- **Pros:** Deterministic guarantee of zero collision errors up to $2^{\text{bit-size}}$.
- **Cons:** Requires keeping track of state across reboots or across distributed cluster instances.

Randomized Approach

- Draw a large nonce uniformly at random for each message (e.g., 96 or 192 bits).
- **Pros:** Stateless generation; works well in distributed systems.
- **Cons:** Subject to the *Birthday Paradox*. If the nonce size is too small (e.g., 64 bits), collisions can happen quickly, breaking security.

Malleability (Lack of Integrity)

A critical concept emphasized by Boneh and Shoup is that stream ciphers provide confidentiality, but offer **zero integrity or authenticity**.

- **The Malleability Flaw:** Because encryption is just bitwise XOR, an active attacker can modify the ciphertext in transit, causing predictable changes in the decrypted plaintext.
- Let $c = m \oplus G(k)$. If the attacker injects a tampering mask Δ:

$$c' = c \oplus \Delta$$

- During decryption, the recipient computes:

$$m' = c' \oplus G(k) = (m \oplus G(k) \oplus \Delta) \oplus G(k) = m \oplus \Delta$$

- The recipient accepts m' as authentic, completely unaware of the modification.

An Example of an Attack

Consider an insecure banking application that encrypts a structured fund transfer instruction using a standard stream cipher:

Field	**Value Layout**
Original Plaintext (m)	`"Transfer $100 to Account 501"`
Target Substring Offset	Bytes 10 to 12 containing `"100"`

- An attacker intercepting this message can construct a mask Δ filled with zeros, except at the target byte offset: $\Delta_{offset} = \texttt{"100"} \oplus \texttt{"999"}$.
- Modifying the ciphertext causes it to decrypt directly to: `"Transfer $999 to Account 501"`.
- **Conclusion:** To prevent this tampering, stream ciphers must always be used alongside authentication primitives like Message Authentication Code (MAC).

From PRGs to PRFs: Why State Matters

- **Limit of PRGs:** A Pseudorandom Generator $G : \{0,1\}^s \to \{0,1\}^n$ scales a short seed into a long, unstructured string. Reading the output requires sequential stream processing.
- Instead of generating a single static string, we want a *keyed function mapping*.
- **Random Access Encryption:** We need to instantly compute the random-looking mask for block x without processing blocks 1 through $x - 1$.
- *A PRF acts like an exponential-sized array of pseudorandom values, where the index is the input x, and the values are computed deterministically on-the-fly using a secret key k.*

Formal Syntax of a Keyed Function

Definition (Keyed Function)

Let $\mathcal{K}$ be the Key Space, $\mathcal{X}$ be the input Space, and $\mathcal{Y}$ be the Range (Output Space). A keyed function F is an efficiently computable deterministic algorithm:

$$F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$$

For a fixed key $k \in \mathcal{K}$, we denote the single-argument function mapping as:

$$F_k(\cdot) = F(k, \cdot) : \mathcal{X} \to \mathcal{Y}$$

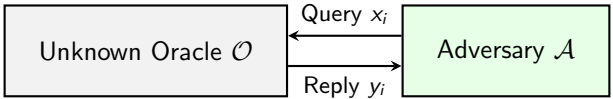
The ideal case: $\mathsf{Funs}[\mathcal{X}, \mathcal{Y}]$

Let $\mathsf{Funs}[\mathcal{X}, \mathcal{Y}]$ denote the set of all possible functions mapping $\mathcal{X}$ to $\mathcal{Y}$.

- Total number of elements: $|\mathsf{Funs}[\mathcal{X}, \mathcal{Y}]| = |\mathcal{Y}|^{|\mathcal{X}|}$.
- If $\mathcal{X} = \{0,1\}^{128}$ and $\mathcal{Y} = \{0,1\}^{128}$, the size is $(2^{128})^{2^{128}} = 2^{128 \cdot 2^{128}}$. This space is huge – a truly random function cannot be stored explicitly.

Defining the Oracle Indistinguishability Model

- We cannot evaluate a PRF by looking at its code. Security must be defined computationally through interaction.
- The Adversary $\mathcal{A}$ is given black-box access to an **Oracle $\mathcal{O}$.**
- $\mathcal{A}$ can feed adaptively chosen inputs $x_i \in \mathcal{X}$ into the box and observe the corresponding outputs $y_i \in \mathcal{Y}$.
- $\mathcal{A}$ does not know the internal configuration of the Oracle.



PRF Attack Game

Let $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ be a keyed function, and let $\mathcal{A}$ be a PPT adversary. PRF Attack Game executes over a random bit challenge b:

1. The Challenger chooses a hidden challenge bit uniformly at random: $b \leftarrow \{0, 1\}$.
2. The Challenger configures the environment oracle $\mathcal{O}$:
   - **If** $b = 1$ **(PRF Case:)** Choose a uniform random key $k \leftarrow \mathcal{K}$. The oracle is assigned to evaluate the real PRF: $\mathcal{O}(\cdot) = F(k, \cdot)$.
   - **If** $b = 0$ **(Random Case:)** Choose a truly random function $g \leftarrow \text{Funs}[\mathcal{X}, \mathcal{Y}]$. The oracle is assigned to evaluate this ideal mapping: $\mathcal{O}(\cdot) = g(\cdot)$.
3. The Adversary $\mathcal{A}$ interacts with $\mathcal{O}$ by issuing adaptive input queries $x_1, x_2, \ldots, x_q$.
4. $\mathcal{A}$ eventually halts and outputs a guess bit $\hat{b} \in \{0, 1\}$.

Winning Condition

The adversary wins PRF Attack Game if $\hat{b} = b$.

In more detail..

Experiment $(b = 1)$ PRF Case

- Key sampled: $k \leftarrow \mathcal{K}$.
- Query response: $y_i = F(k, x_i)$.
- Queries with the same input x_i will naturally yield identical deterministic responses.

$$\Pr[\mathcal{A}^{F(k,\cdot)} = 1]$$

Experiment $(b = 0)$ Random Case

- Function sampled: $g \leftarrow \text{Funs}[\mathcal{X}, \mathcal{Y}]$.
- Query response: $y_i = g(x_i)$.
- If x_i was never queried before, y_i is sampled completely fresh from $\mathcal{Y}$. If x_i was queried prior, the oracle repeats the saved choice.

$$\Pr[\mathcal{A}^{g(\cdot)} = 1]$$

Mathematical Advantage Formulation

An adversary can achieve a winning probability of 50% simply by guessing blindly. We therefore isolate how effectively $\mathcal{A}$ alters its behavior between the two distinct worlds.

Definition (PRF Advantage)

The mathematical advantage of an adversary $\mathcal{A}$ against a keyed function F in PRF Attack Game is defined as:

$$\mathbf{Adv}_{\mathrm{PRF}}[F, \mathcal{A}] := \left| \Pr[\hat{b} = 1 \mid b = 1] - \Pr[\hat{b} = 1 \mid b = 0] \right|$$

Expanding this utilizing our operational oracles results in the core formulation:

$$\mathbf{Adv}_{\mathrm{PRF}}[F, \mathcal{A}] = \left| \Pr_{k \leftarrow \mathcal{K}}[\mathcal{A}^{F(k,\cdot)} = 1] - \Pr_{g \leftarrow \mathrm{Funs}[\mathcal{X}, \mathcal{Y}]}[\mathcal{A}^{g(\cdot)} = 1] \right|$$

Defining Computational PRF Security

Asymptotic Definition

A keyed function $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ is a secure PRF if for all Probabilistic Polynomial-Time (PPT) adversaries $\mathcal{A}$, the advantage function is negligible with respect to the security parameter λ:

$$\mathbf{Adv}_{\mathsf{PRF}}[F, \mathcal{A}] \leq \mathit{negl}(\lambda)$$

From Functions to Permutations

- **The Core Limitation of PRFs:** A PRF $F_k : \mathcal{X} \to \mathcal{Y}$ is not guaranteed to be invertible. For symmetric encryption (block ciphers), decryption requires a strict *bijective mapping* where $\mathcal{X} = \mathcal{Y}$.
- **Syntax of a Keyed Permutation:** Let $E : \mathcal{K} \times \mathcal{X} \to \mathcal{X}$ be an efficiently computable deterministic algorithm. For a fixed key $k \in \mathcal{K}$, $E_k(\cdot) = E(k, \cdot)$ is a bijection.
- **Invertibility Requirement:** There exists an efficient decryption algorithm $D : \mathcal{K} \times \mathcal{X} \to \mathcal{X}$ such that for all $k \in \mathcal{K}$ and all $x \in \mathcal{X}$:

$$D(k, E(k, x)) = x$$

.

The Ideal Permutation Space: Perms$[\mathcal{X}]$

Let Perms$[\mathcal{X}]$ denote the set of all possible permutations on $\mathcal{X}$.

- Total size: $|\text{Perms}[\mathcal{X}]| = (|\mathcal{X}|)!$
- A truly random permutation $P \leftarrow_R \text{Perms}[\mathcal{X}]$ chooses a bijective mapping without any internal algebraic structure.

PRP Attack Game (PRP Indistinguishability)

The security of a PRP is evaluated using an oracle indistinguishability game where the ideal world is populated by a truly random permutation instead of a random function.

1. The Challenger chooses a hidden challenge bit uniformly at random: $b \leftarrow \{0,1\}$.
2. The Challenger configures the environment oracle $\mathcal{O}$:
   - **If $b = 1$ (PRP Case):** Choose a uniform random key $k \leftarrow \mathcal{K}$. Set $\mathcal{O}(\cdot) = E(k, \cdot)$.
   - **If $b = 0$ (Random Permutation Case):** Choose a truly random permutation $P \leftarrow_R \text{Perms}[\mathcal{X}]$. Set $\mathcal{O}(\cdot) = P(\cdot)$.
3. $\mathcal{A}$ makes at most q adaptive queries to $\mathcal{O}(\cdot)$ and outputs a guess bit $\hat{b} \in \{0,1\}$.

Definition (PRP Advantage)

The advantage of $\mathcal{A}$ against a keyed permutation E in the above attack game is:

$$\mathbf{Adv}_{\text{PRP}}[E, \mathcal{A}] := \left| \Pr_{k \leftarrow \mathcal{K}}[\mathcal{A}^{E(k,\cdot)} = 1] - \Pr_{P \leftarrow \text{Perms}[\mathcal{X}]}[\mathcal{A}^{P(\cdot)} = 1] \right|$$

The Birthday Paradox

The Counter-Intuitive Reality

- **The Question:** How many people do you need in a room for a 50% chance that at least two share a birthday?
- **Intuitive Guess:** Around 183 (365/2).
- **Mathematical Fact:** You only need **23 people**.

Why Intuition Fails: Pairs vs. Individuals

- We naturally think about the odds of someone sharing *our* specific birthday. The paradox counts the number of **distinct pairs** that can be formed.
- With q people, the number of pairs grows quadratically: $\binom{q}{2} = \frac{q(q-1)}{2}$.

The probability $P(\text{collision})$ that at least two people share a birthday out of N possible days (where $N = 365$) is roughly:

$$P(\text{collision}) \approx 1 - e^{-\frac{q(q-1)}{2N}} \approx 1 - e^{-\frac{q^2}{2N}}$$

When $q = 23$ and $N = 365$, $P(\text{collision}) \approx 50.7\%$.

The Fundamental Dilemma

- In many cryptographic proofs, it is significantly easier to analyze security assuming a primitive is a PRF (Funs$[\mathcal{X}, \mathcal{X}]$) because output values are completely independent.
- However, practical block ciphers (AES) are PRPs (Perms$[\mathcal{X}]$) because they cannot allow collisions under a fixed key.
- **The Question:** How much advantage does it gain from the mere fact that a permutation never repeats an output value?
- *The PRP/PRF Switching Lemma* establishes a strict information-theoretic boundary between a truly random function and a truly random permutation.

PRP/PRF Switching Lemma Theorem

Theorem (The Switching Lemma)

Let $\mathcal{X}$ be a finite set, and let $\mathcal{A}$ be an adversary that makes at most q distinct queries to its oracle. Then:

$$\left| \Pr_{g \leftarrow Funs[\mathcal{X},\mathcal{X}]}[\mathcal{A}^{g(\cdot)} = 1] - \Pr_{P \leftarrow Perms[\mathcal{X}]}[\mathcal{A}^{P(\cdot)} = 1] \right| \leq \frac{q(q-1)}{2|\mathcal{X}|} \leq \frac{q^2}{2|\mathcal{X}|}$$

- This bound is completely independent of the adversary's computational time or strategy. It is purely information-theoretic.
- If $|\mathcal{X}| = 2^{128}$ (AES block size), an adversary making $q = 2^{40}$ queries cannot distinguish a true permutation from a true function with an advantage greater than $\frac{2^{80}}{2^{129}} = 2^{-49}$.

Proof via Games

We formalize the proof by designing two identical systems that only diverge when a specific event occurs. Let $\mathcal{A}$ make q distinct queries $x_1, \ldots, x_q$.

Simulating Funs$[\mathcal{X}, \mathcal{X}]$ via Lazy Evaluation

When $\mathcal{A}$ queries x_i, sample $y_i \leftarrow_R \mathcal{X}$.

Simulating Perms$[\mathcal{X}]$ via Lazy Evaluation

When $\mathcal{A}$ queries x_i, sample $y_i \leftarrow_R \mathcal{X} \setminus \{y_1, \ldots, y_{i-1}\}$.

Let Coll be the event that a random function samples an output value that it has already used before (a collision in the execution history):

$$\text{Coll} \iff \exists\, i \neq j \text{ such that } y_i = y_j$$

Note that *as long as the event Coll does not occur, the conditional output distributions of both systems are completely identical.* Therefore:

$$\left|\Pr[\mathcal{A}^{\text{Funs}} = 1] - \Pr[\mathcal{A}^{\text{Perms}} = 1]\right| \leq \Pr[\text{Coll}]$$

Proof Bounding the Collision

Now we calculate the exact probability of the event Coll occurring in a random function execution across q queries.

- For any two distinct queries x_i and x_j, the outputs y_i and y_j are picked uniformly and independently from $\mathcal{X}$.
- The probability that these two specific points collide is exactly:

$$\Pr[y_i = y_j] = \frac{1}{|\mathcal{X}|}$$

- The total number of unique pairs (i, j) among q queries is given by the binomial coefficient:

$$\binom{q}{2} = \frac{q(q-1)}{2}$$

Applying the Union Bound across all possible pairs yields:

$$\Pr[\text{Coll}] \leq \sum_{1 \leq i < j \leq q} \Pr[y_i = y_j] = \binom{q}{2} \frac{1}{|\mathcal{X}|} = \frac{q(q-1)}{2|\mathcal{X}|}$$

This completes the formal proof of the Switching Lemma.

The Standard PRF-PRP Substitution Rule

How do we use this in proofs? We use it to switch from a concrete block cipher to an ideal random function using a triangle inequality chain.

Advantage Breakdown Chain

$$\mathbf{Adv}_{\mathrm{PRF}}[E, \mathcal{A}] \leq \mathbf{Adv}_{\mathrm{PRP}}[E, \mathcal{A}] + \frac{q^2}{2|\mathcal{X}|}$$

- **Interpretation:** If an adversary can successfully break a block cipher modeled as a PRF, they must either:
  1. Actually break the underlying block cipher engineering as a PRP ($\mathbf{Adv}_{\mathrm{PRP}}[E, \mathcal{A}]$ is high).
  2. Make enough queries to exploit natural birthday boundary limitations ($\frac{q^2}{2|\mathcal{X}|}$ becomes large).
     - In a distinguishing attack, if the oracle is a PRF, by the birthday paradox, when q approaches $2^{n/2}$, it is highly likely that at least two of the outputs will be identical ($y_i = y_j$).
- This ensures that any secure block cipher can safely replace a random function in any high-level cryptographic construction up to the Birthday Bound limit.

Stream Cipher Evolution: From LFSRs and RC4 to ChaCha20

Legacy Insecure Constructions

- **LFSRs (Linear Feedback Shift Registers):** Highly predictable due to inherent linearity. Easily broken using the Berlekamp-Massey algorithm.
- **RC4:** Popular software cipher plagued by severe byte-biases (e.g., the second byte bias). It leaks keystream information, leading to its complete ban in TLS.

Modern Secure Design: ChaCha20

- **ARX Architecture:** Built entirely on simple hardware-friendly operations: **A**ddition, **R**otation, and e**X**clusive-OR (XOR).
- **State and Speed:** Operates on a 4×4 matrix of 32-bit words (512-bit state). Eliminates data-dependent lookups to offer native immunity against cache-timing side-channel attacks.

Block Cipher Architectures: Feistel Networks vs. SPNs

Feistel Networks (e.g., DES)

- **Mechanism:** Splits the input block into left (L_i) and right (R_i) halves. The round update rule follows: $L_{i+1} = R_i$ and $R_{i+1} = L_i \oplus F(R_i, K_i)$.
- **Core Advantage:** The round function F does *not* need to be invertible. The overall Feistel structure guarantees invertibility for decryption.

Substitution-Permutation Networks (e.g., AES)

- **Mechanism:** Processes the entire state in parallel using layered mathematical steps. Every single operation inside an SPN **must be invertible**.
- **Shannon's Principles for Security:**
  - **Confusion (S-Box):** Non-linear byte substitution to obscure local relationships.
  - **Diffusion (ShiftRows/MixColumns):** Permutation layers to spread local changes rapidly across the entire block state.